

Copy and fill for mdspan

Document #: P3242R1
Date: 2025-03-01
Project: Programming Language C++
LEWG
Reply-to: Nicolas Morales
<nmmoral@sandia.gov>
Christian Trott
<crtrott@sandia.gov>
Mark Hoemmen
<mark.hoemmen@gmail.com>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>

Contents

- 1 Revision History
 - 1.1 P3242R1: 2025-03 Post-Hagenberg Mailing
 - 1.2 P3242R0: 2024-04 Post-Tokyo Mailing
- 2 Motivation
 - 2.1 Safety
- 3 Design
 - 3.1 Header
 - 3.2 Existing copy in `std::linalg`
 - 3.3 What the proposal does not include
- 4 Wording
- 5 Acknowledgments
- 6 References

§ 1 Revision History

§ 1.1 P3242R1: 2025-03 Post-Hagenberg Mailing

- Add discussion of applying copy to `mdspans` of differing sizes

§ 1.2 P3242R0: 2024-04 Post-Tokyo Mailing

— Initial paper revision

§ 2 Motivation

C++23 introduced `mdspan` ([P0009R18]), a non-owning multidimensional array abstraction that has a customizable layout. Layout customization was originally motivated in [P0009R18] with considerations for interoperability and performance, particularly on different architectures. Moreover, [P2630R4] introduced `submdspan`, a slicing function that can yield arbitrarily strided layouts. However, without standard library support, copying efficiently between `mdspans` with mixes of complex layouts is challenging for users.

Many applications, including high-performance computing (HPC), image processing, computer graphics, etc that benefit from `mdspan` also would benefit from basic memory operations provided in standard algorithms such as `copy` and `fill`. Indeed, the authors found that a `copy` algorithm would have been quite useful in their implementation of the `mdarray` ([P1684R5]) constructor. A more constrained form of `copy` is also included in the standard linear algebra library ([P1673R13]).

However, existing standard library facilities are not sufficient here. Currently, `mdspan` does not have iterators or ranges that represent the span of the `mdspan`. Additionally, it's not entirely clear what this would entail. `std::linalg::copy` ([P1673R13]) is limited to `mdspans` of rank 2 or lower.

Moreover, the manner in which an `mdspan` is copied (or filled) is highly performance sensitive, particularly in regards to caching behavior when traversing `mdspan` memory. A naive user implementation is easy to get wrong in addition to being tedious for higher rank `mdspans`. Ideally, an implementation would be free to use information about the layout of the `mdspan` known at compile time to perform optimizations; e.g. a continuous span `mdspan` copy for trivial types could be implemented with a `memcpy`.

Finally, providing these generic algorithms would also enable these operations for types that are representable by `mdspan`. For example, this would naturally include `mdarray`, which is convertible to `mdspan`, or for user-defined types whose view of memory corresponds to `mdspans` (e.g. an image class or something similar).

§ 2.1 Safety

Due to the closed nature of `mdspan` extents, `copy` operations can be checked by the implementation to prevent past-the-end writes. This is an advantage the proposed `copy` operation has over the existing operations in the standard.

§ 3 Design

The main design direction of this proposal is to provide methods for copying and filling `mdspan`s that may have differing layouts and accessors, while allowing implementations to provide efficient implementations for special cases. For example, if a copy occurs between two `mdspan`s with the same layout mapping type that is contiguous and both use `default_accessor`, the intention is that this could be implemented by a single `memcpy`.

Furthermore, accessors as a customization point should be enabled, as with any other `mdspan` operation. For example, a custom accessor that checks a condition inside of the access method should still work and check that condition. It's worth noting that there may be a high sensitivity of how much implementations able to optimize if provided custom accessors. For example, optimizations could be disabled if using a custom accessor that is identical to the default accessor.

A common question about copy to `mdspan` is whether it should support `mdspan`s with differing extent (and presumably applying the operation only to the overlapping part). It's the authors' firm opinion that copy should **not** support this. Users wishing to copy an `mdspan` of a differing size should instead make use of `submdspan` [P2630R4] as this provides explicit control of which subregions of the `mdspan` the user actually desires to copy.

Finally, there is some question as to whether `copy` and `fill` should return a value when applied to `mdspan`, as the iterator and ranged-based algorithms do. We believe that `mdspan` `copy` and `fill` should return `void`, as there is no past-the-end iterator that they could reasonably return.

§ 3.1 Header

Currently, we are proposing adding `copy` and `fill` algorithms on `mdspan` to header `<mdspan>`. We considered other options, namely:

- `<algorithm>`: This would mean that users of iterator-based algorithms would need to pull in `<mdspan>`. On the other hand, this is where iterator-based `copy` and `fill` live so may be preferable in that sense.
- `<mdspan_algorithm>` (or similarly any other new header): This seems like overkill for two functions. However, in the future, we may want to add new algorithms for `mdspan` that are not strictly covered by existing algorithms in `<algorithm>`, so this option may be more future proof.

We settled on `<mdspan>` because as proposed this is a relatively light-weight addition that reflects operations that are commonly desired with `mdspan`s. However, the authors are open to changing this.

§ 3.2 Existing copy in `std::linalg`

[P1673R13] introduced several linear algebra operations including `std::linalg::copy`. This operation only applies to `mdspan`s with $rank \leq 2$. This paper is proposing a version of

copy that is not constrained by the number of ranks and differs from `std::linalg::copy` in some important ways outline below.

Note that right now the strict addition of `copy` would potentially cause the following code to be ambiguous, due to ADL-finding `std::copy`:

```
using std::linalg::copy;
copy(mds1, mds2);
```

One possibility would be to remove `std::linalg::copy`, as it is a subset of the proposed `std::copy`. This was rejected by the paper authors because of certain requirements in `[linalg.reqs.alg]` – that is:

The function may make arbitrarily many objects of any linear algebra value type, value-initializing or direct-initializing them with any existing object of that type.

This requirement is likely undesirable for a generalized copy algorithm.

There is a similar argument against simply generalizing `std::linalg::copy`. In addition to the freedom of `std::linalg::copy` to arbitrarily value or direct-initializing values, using the linear algebra version of `copy` would require the use of unnecessary includes and namespaces. It seems not very ergonomic for a user to have to use `std::linalg::copy` and include `<linalg>` even if the `mdspan` operations they are performing are unrelated to linear algebra.

§ 3.3 What the proposal does not include

There are a few additions that are analogous to existing standard algorithms that are not included in this proposal, both to keep the proposal small and because some of these algorithms do not make sense in the context of `mdspan`:

- `std::move`: Perhaps this should be included for completeness's sake. However, it doesn't seem applicable to the typical usage of `mdspan`.
- `(copy|fill)_n`: As a multidimensional view `mdspan` does not in general follow a specific ordering. Memory ordering may not be obvious to calling code, so it's not even clear how these would work. Any applications intending to copy a subset of `mdspan` should use call `copy` on the result of `submdspan`.
- `copy_backward`: As above, there is no specific ordering. A similar effect could be achieved via transformations with a custom layout, similar to `layout_t::transpose` in [\[P1673R13\]](#).
- Other algorithms, include `std::for_each`. `for_each` in particular is a desirable but brings in many unanswered questions that should be addressed in a different paper.

§ 4 Wording

```

template<class SrcElementType, class SrcExtents, class SrcLayoutPolicy,
        class SrcAccessorPolicy,
        class DstElementType, class DstExtents, class DstLayoutPolicy,
        class DstAccessorPolicy>
void copy(mdspan<SrcElementType, SrcExtents, SrcLayoutPolicy,
            SrcAccessorPolicy> src,
        mdspan<DstElementType, DstExtents, DstLayoutPolicy,
            DstAccessorPolicy> dst);

template<class ExecutionPolicy,
        class SrcElementType, class SrcExtents, class SrcLayoutPolicy,
        class SrcAccessorPolicy,
        class DstElementType, class DstExtents, class DstLayoutPolicy,
        class DstAccessorPolicy>
void copy(ExecutionPolicy&& policy,
        mdspan<SrcElementType, SrcExtents, SrcLayoutPolicy,
            SrcAccessorPolicy> src,
        mdspan<DstElementType, DstExtents, DstLayoutPolicy,
            DstAccessorPolicy> dst);

```

1 Constraints:

(1.1) — `std::is_assignable_v<typename mdspan<SrcElementType, SrcExtents, SrcLayoutPolicy, SrcAccessorPolicy>::reference, typename mdspan<DstElementType, DstExtents, DstLayoutPolicy, DstAccessorPolicy>::reference>` is true.

(1.2) — `is_constructible_v<DstExtents, SrcExtents>`.

2 Preconditions:

(2.1) — `src.extents() == dst.extents()` is true.

(2.2) — `dst.is_unique()` is true.

(2.3) — there is no unique multidimensional index `i...` in `src.extents()` where there exists a multidimensional index `j...` in `dst.extents()` such that `src[i...]` and `dst[j...]` refer to the same element.

3 *Effects:* for all unique multidimensional indices `i...` in `src.extents()`, assigns `src[i...]` to `dst[i...]`

```

template<class ElementType, class Extents, class LayoutPolicy, class
        AccessorPolicy, class T>
void fill(mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy> dst,
        const T& value);

template<class ExecutionPolicy,
        class ElementType, class Extents, class LayoutPolicy, class
        AccessorPolicy, class T>
void fill(ExecutionPolicy&& policy,
        mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy> dst,
        const T& value);

```

- 4 *Constraints:* `std::is_assignable_v<typename mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>::reference, const T&>`
- 5 *Effects:* for all unique multidimensional indices `i...` in `dst.extents()`, assigns value to `dst[i...]`

§ 5 Acknowledgments

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

§ 6 References

[P0009R18]

Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. 2022. `mdspan`.

<https://wg21.link/p0009r18>

[P1673R13]

Mark Hoemmen, Daisy Hollman, Christian Trott, Daniel Sunderland, Nevin Liber, Alicia Klinvex, Li-Ta Lo, Damien Lebrun-Grandie, Graham Lopez, Peter Caday, Sarah Knepper, Piotr Luszczek, Timothy Costa. 2023. A free function linear algebra interface based on the BLAS.

<https://wg21.link/p1673r13>

[P1684R5]

Christian Trott, Daisy Hollman, Mark Hoemmen, Daniel Sunderland, Damien Lebrun-Grandie. 2023. `mdarray`: An Owning Multidimensional Array Analog of `mdspan`.

<https://wg21.link/p1684r5>

[P2630R4]

Christian Trott, Damien Lebrun-Grandie, Mark Hoemmen, Nevin Liber. 2023. `Submdspan`.

<https://wg21.link/p2630r4>